

GoF Design Patterns

23 Gang of Four Design Patterns — concise study notes in GoF order

Difficulty scale: 1/10 = easiest to understand initially, 10/10 = hardest conceptually. Difficulty is subjective and reflects the conceptual learning curve, not implementation length.

Creational Patterns

1. Abstract Factory (7/10)

What exactly does it do? Create families of related objects without specifying their concrete classes.

How it works: A factory interface exposes methods such as `createButton()` and `createCheckbox()`. Concrete factories create a consistent product family, such as Windows UI or macOS UI.

When to use: Use when the application must work with interchangeable families of related products and consistency between products matters.

Core idea: Key idea: one factory creates multiple related product types.

2. Builder (5/10)

What exactly does it do? Construct a complex object step-by-step while separating construction from the final representation.

How it works: A Builder exposes construction steps such as `buildWalls()`, `buildRoof()`, and `buildGarage()`, then `build()` returns the finished object.

When to use: Use when an object has many optional parts, complicated construction steps, or several valid representations.

Core idea: Key idea: build a complex object gradually.

3. Factory Method (6/10)

What exactly does it do? Define an object-creation method while allowing subclasses to decide which concrete object is created.

How it works: A superclass contains the overall workflow and calls a factory method. Subclasses override that method to create their product.

When to use: Use when a superclass needs to work with products abstractly but subclasses should control product creation.

Core idea: Key idea: subclass decides the concrete product.

4. Prototype (4/10)

What exactly does it do? Create new objects by cloning an existing prototype.

How it works: A prototype contains a configured object. New objects are produced by `clone()`, avoiding repeated expensive or complicated initialization.

When to use: Use when creating an object from scratch is expensive or when many similar objects are needed.

Core idea: Key idea: create by copying an existing object.

5. Singleton (3/10)

What exactly does it do? Ensure a class has only one instance and provide a global access point to it.

How it works: The class controls its own construction and exposes an access method such as `getInstance()`.

When to use: Use only when exactly one shared instance is genuinely required. It introduces global state and can make testing harder.

Core idea: Key idea: one controlled instance.

Structural Patterns

6. Adapter (3/10)

What exactly does it do? Convert one interface into another interface expected by the client.

How it works: The Adapter wraps an incompatible class and translates calls. The client uses the target interface without knowing about the adaptee's incompatible API.

When to use: Use when existing code works correctly but its interface does not match the interface your application expects.

Core idea: Key idea: make incompatible interfaces work together.

7. Bridge (7/10)

What exactly does it do? Separate an abstraction from its implementation so the two can vary independently.

How it works: Instead of combining every abstraction with every implementation, the abstraction contains a reference to an implementation interface.

When to use: Use when there are two independent dimensions of variation and inheritance would create a combinatorial explosion of classes.

Core idea: Key idea: separate two dimensions that change independently.

8. Composite (5/10)

What exactly does it do? Compose objects into tree structures and treat individual objects and groups uniformly.

How it works: A common Component interface is implemented by Leaf and Composite. Composite contains children and forwards operations recursively.

When to use: Use for menus, file systems, organization trees, GUI hierarchies, and other part-whole structures.

Core idea: Key idea: treat one and many uniformly.

9. Decorator (4/10)

What exactly does it do? Attach additional responsibilities to an object dynamically without changing its class.

How it works: Decorators implement the same interface as the wrapped object and delegate to it while adding behavior before or after the delegation.

When to use: Use when features should be combined dynamically and subclassing would create too many combinations.

Core idea: Key idea: wrap an object to add behavior.

10. Facade (2/10)

What exactly does it do? Provide a simple interface to a complicated subsystem.

How it works: The Facade coordinates several subsystem classes behind a small number of high-level operations.

When to use: Use when clients should not need to understand or coordinate many low-level subsystem classes.

Core idea: Key idea: simplify access to complexity.

11. Flyweight (8/10)

What exactly does it do? Share common object data to support large numbers of fine-grained objects efficiently.

How it works: Separate intrinsic state, which can be shared, from extrinsic state, which is supplied by the client. A Flyweight factory reuses shared objects.

When to use: Use when huge numbers of similar objects consume too much memory and much of their state can be shared.

Core idea: Key idea: share common state to reduce memory.

12. Proxy (5/10)

What exactly does it do? Provide a substitute or intermediary for another object to control access to it.

How it works: The Proxy implements the same interface as the real subject and can add lazy loading, access control, caching, remote access, or logging.

When to use: Use when access to the real object needs control or extra behavior.

Core idea: Key idea: stand in front of the real object.

Behavioral Patterns

13. Chain of Responsibility (4/10)

What exactly does it do? Pass a request along a chain of handlers until a handler handles it.

How it works: Each handler decides whether it can process the request. If not, it forwards the request to the next handler.

When to use: Use when several objects may handle a request and the sender should not know which object will handle it.

Core idea: Key idea: request travels through handlers.

14. Command (5/10)

What exactly does it do? Encapsulate a request or action as an object.

How it works: A Command exposes execute() and contains a reference to a Receiver that performs the real operation. Commands can be queued, logged, scheduled, or undone.

When to use: Use when operations need to be parameterized, stored, queued, logged, or undone.

Core idea: Key idea: turn an action into an object.

15. Interpreter (9/10)

What exactly does it do? Define a representation of a grammar and interpret sentences in that language.

How it works: Expressions form an object structure such as terminal and non-terminal expressions. Each expression interprets part of the input.

When to use: Use for small, simple domain-specific languages or rule systems. It becomes unwieldy for large grammars.

Core idea: Key idea: represent and evaluate a small language.

16. Iterator (3/10)

What exactly does it do? Provide sequential access to elements of a collection without exposing its internal representation.

How it works: An Iterator commonly provides `hasNext()` and `next()`. The collection supplies an iterator appropriate to its internal structure.

When to use: Use when clients need traversal independent of whether the collection is an array, list, tree, or another structure.

Core idea: Key idea: traverse without knowing collection internals.

17. Mediator (7/10)

What exactly does it do? Encapsulate how a set of objects interact so they do not depend directly on one another.

How it works: Colleagues communicate through a Mediator. The Mediator coordinates the interaction and reduces many-to-many dependencies.

When to use: Use when many objects communicate in complicated ways and direct references create tight coupling.

Core idea: Key idea: centralize communication.

18. Memento (6/10)

What exactly does it do? Capture and externalize an object's state so it can later be restored without exposing implementation details.

How it works: The Originator creates a Memento containing its state. A Caretaker stores mementos and later gives one back to the Originator for restoration.

When to use: Use for undo/redo, checkpoints, snapshots, and rollback.

Core idea: Key idea: save and restore state.

19. Observer (4/10)

What exactly does it do? Define a one-to-many dependency so observers are notified when a subject changes state.

How it works: Observers register with a Subject. When the Subject changes, it notifies its observers.

When to use: Use for event systems, GUI notifications, publish-subscribe style relationships, and model-view updates.

Core idea: Key idea: one subject notifies many dependents.

20. State (7/10)

What exactly does it do? Allow an object to alter its behavior when its internal state changes.

How it works: State objects encapsulate state-specific behavior. The Context delegates operations to its current State.

When to use: Use when conditional logic based on state becomes large, repetitive, or difficult to maintain.

Core idea: Key idea: state determines behavior.

21. Strategy (4/10)

What exactly does it do? Define a family of interchangeable algorithms and make them selectable independently from the client.

How it works: The Context holds a Strategy interface. Concrete strategies implement different algorithms and can be substituted at runtime.

When to use: Use when several algorithms solve the same problem and the client should be able to choose among them.

Core idea: Key idea: encapsulate interchangeable algorithms.

22. Template Method (6/10)

What exactly does it do? Define the skeleton of an algorithm in a superclass while allowing subclasses to redefine selected steps.

How it works: The template method fixes the overall sequence. Primitive operations are overridden by subclasses; hooks can optionally alter behavior.

When to use: Use when several classes share an algorithm structure but differ in some steps.

Core idea: Key idea: fixed algorithm skeleton, variable steps.

23. Visitor (9/10)

What exactly does it do? Represent an operation to be performed on elements of an object structure, allowing new operations without changing the element classes.

How it works: Elements provide accept(visitor). The Visitor contains operations for each concrete element type. Double dispatch helps select the appropriate visit method.

When to use: Use when the object structure is relatively stable but many new operations must be added.

Core idea: Key idea: add operations separately from the object structure.

Quick Comparison

Pattern	Remember it as
Abstract Factory	Create related families
Builder	Build complex object step-by-step
Factory Method	Subclass decides product
Prototype	Clone existing object
Singleton	One controlled instance
Adapter	Convert interface
Bridge	Separate abstraction/implementation
Composite	Treat one and many uniformly
Decorator	Wrap to add behavior
Facade	Simplify subsystem
Flyweight	Share common state
Proxy	Control access
Chain of Responsibility	Pass request along handlers
Command	Action becomes object
Interpreter	Interpret a small language
Iterator	Traverse collection
Mediator	Centralize communication
Memento	Save/restore state
Observer	Notify dependents
State	State changes behavior
Strategy	Interchangeable algorithms
Template Method	Algorithm skeleton
Visitor	Add operations separately